Week 7 Report: Cost Optimization and Continuous Learning

1. Overview

This week I added cost optimization and a feedback loop on top of my Week 5/6 TechCorp agent. All the new work is in `cost_optimization_starter.py`, which holds three independent classes:

- **CostAnalyzer** — tracks the cost of each query, breaks the daily total down by component, and flags statistically expensive queries.
- **OptimizationStrategy** — cuts cost through caching, retrieval top-k, complexity-aware model selection, and response compression.
- **FeedbackLoop** — collects user corrections, validates them by role authority, and measures their quality over time.

Following the assignment, the rest of `week7/` is self-contained: I copied my Week 6 `app_starter.py` and `access_control_starter.py` into the folder and did not modify my earlier weeks. The three new classes operate on cost dictionaries and plain strings, so the whole test block runs offline on synthetic data and spends no LLM quota. The agent in `app_starter.py` already produces exactly the cost dictionaries `CostAnalyzer` consumes, so it can be fed real queries without any change.

2. CostAnalyzer

`record_query` normalizes each entry: missing cost components default to zero, `total_cost` is recomputed from the four components so it can never drift out of sync, and a UTC timestamp is stamped. `get_cost_breakdown` returns the per-component totals (retrieval, LLM, tool, error), the daily total, and the query count.

`identify_cost_spikes` flags any query whose total cost exceeds `mean + 2*stdev` of all recorded costs. One detail worth noting: with very few queries a single extreme outlier inflates the standard deviation past its own value, so the rule finds nothing. That is expected behavior for this formula, not a bug — it needs a realistic baseline of queries to work. My test feeds ten cheap baseline queries plus one retry-storm query ($0.052 versus ~$0.001), and the spike is correctly isolated above the $0.036 threshold.

3. OptimizationStrategy

- **Caching** — an exact-match cache returns a stored answer on a repeat query, avoiding the LLM call entirely. Hits and misses are counted.
- **Retrieval top-k** — `optimize_retrieval_count` caps retrieved documents at 3 (e.g. 15 → 3), cutting input tokens, while never dropping to zero when documents exist.
- **Model selection** — `select_model_by_complexity` routes simple questions to the cheap model (`gemini-3.5-flash`, the tier my agent settled on in Week 5) and complex ones (analyze / compare / design, or very long prompts) to `gemini-2.5-pro`.

- **Response compression** — long answers are trimmed to their first three sentences; short answers pass through untouched.

`get_optimization_impact` estimates combined savings. I combine per-strategy estimates multiplicatively rather than adding them, so the total can never exceed 100%. Caching is treated honestly: instead of a fixed guess, its saving is the measured cache hit rate. With all four strategies active and a 50% hit rate, the estimated saving is about 87%.

4. FeedbackLoop

There are two gates by design. The **acceptance** gate (in `submit_correction`) lets anyone with a recognized role contribute a correction, as long as the correction is more detailed than the original answer; unknown roles and non-improving edits are rejected and not stored. The stricter **validation** gate (in `validate_correction`) only trusts corrections from manager level and above (authority level ≥ 3) that genuinely differ from and extend the original. This mirrors a real system: everyone can flag a mistake, but only a sufficiently authoritative, substantive correction is trusted enough to act on.

`get_feedback_metrics` reports the number of stored corrections, the validation rate, the average correction length, the count of rejected submissions, and the top recurring error topics (travel, compensation, leave, expense, security) derived from the corrected queries.

5. Tests

The `__main__` block is a self-checking test harness covering all three classes with positive, negative, and edge cases — empty state, a cache hit after a miss, retrieval pruning bounds, model routing both ways, compression of long versus short answers, an engineer correction that is accepted but not validated, a manager correction that is both, and rejections for an unknown role and a non-detailed edit. It prints PASS/FAIL per case and exits non-zero if anything fails.

All 29 checks pass:

```
❯ python3 cost_optimization_starter.py

=== Testing CostAnalyzer ===
  [PASS] empty breakdown totals are zero
  [PASS] no spikes with no data
INFO:__main__:Recorded query (total=$0.001300): What is the PTO policy?
INFO:__main__:Recorded query (total=$0.001000): Where is the office?
INFO:__main__:Recorded query (total=$0.001200): What is the dress code?
INFO:__main__:Recorded query (total=$0.001400): How many vacation days?
INFO:__main__:Recorded query (total=$0.001100): What time does the office open?
INFO:__main__:Recorded query (total=$0.001000): Is there a gym?
INFO:__main__:Recorded query (total=$0.001500): What is the parental leave policy?
INFO:__main__:Recorded query (total=$0.000900): How do I reset my password?
INFO:__main__:Recorded query (total=$0.001400): Where do I submit expenses?
INFO:__main__:Recorded query (total=$0.001400): What is the remote work policy?
INFO:__main__:Recorded query (total=$0.052000): Compare every benefit across all 7 departments and summarize
  breakdown: {
  "retrieval_total": 0.0065,
  "llm_total": 0.0395,
  "tool_total": 0.0032,
  "error_total": 0.015,
  "query_count": 11,
  "total_daily": 0.0642
}
  [PASS] query_count is 11
  [PASS] total_daily matches manual sum
  [PASS] components sum to total
WARNING:__main__:Detected 1 cost spike(s) above $0.036460
  spikes: [
  {
    "query_text": "Compare every benefit across all 7 departments and summarize",
    "total_cost": 0.052,
    "threshold": 0.03646044,
    "mean": 0.00583636,
    "stdev": 0.01531204,
    "times_over_mean": 8.91
  }
]
  [PASS] exactly one spike detected
  [PASS] spike is the expensive comparison query
```

```
=== Testing OptimizationStrategy ===
INFO:__main__:Cache MISS, stored query: What is the PTO policy?
INFO:__main__:Cache HIT for query: What is the PTO policy?
  [PASS] first lookup is a miss
  [PASS] second lookup is a hit
INFO:__main__:Retrieval pruned: 15 -> 3 docs
  [PASS] 15 docs pruned to 3
  [PASS] 2 docs left as 2
  [PASS] 0 docs stays 0
INFO:__main__:Selected model gemini-3.5-flash for query: What is the office address?
  [PASS] simple query -> cheap model
INFO:__main__:Selected model gemini-2.5-pro for query: Analyze and compare the travel policy across departments
  [PASS] complex query -> capable model
INFO:__main__:Compressed response: 5 -> 3 sentences
  [PASS] compression keeps 3 sentences
  [PASS] short answer passes through unchanged
  impact: {
  "total_savings_pct": 87.2,
  "strategies_applied": [
    "caching",
    "retrieval_pruning",
    "model_selection",
    "response_compression"
  ],
  "breakdown": {
    "retrieval_pruning": "40%",
    "model_selection": "50%",
    "response_compression": "15%",
    "caching": "50%"
  },
  "cache_hit_rate": 0.5
}
  [PASS] impact lists applied strategies
  [PASS] total savings between 0 and 100
  [PASS] cache hit rate reported

=== Testing FeedbackLoop ===
INFO:__main__:Accepted correction from engineer for query: What is the travel policy for flights over 8 hours?
  [PASS] engineer correction accepted
  [PASS] engineer correction not validated (below manager)
INFO:__main__:Accepted correction from manager for query: How many vacation days do new hires get?
  [PASS] manager correction accepted
  [PASS] manager correction validated
  [PASS] unknown role rejected
  [PASS] non-detailed correction rejected
  metrics: {
  "total_corrections": 2,
  "validation_rate": 0.5,
  "avg_correction_length": 83.5,
  "top_error_patterns": [
    {
      "pattern": "travel",
      "count": 1
    },
    {
      "pattern": "leave",
      "count": 1
    }
  ],
  "rejected_submissions": 2
}
  [PASS] two corrections stored
  [PASS] validation rate is 0.5
  [PASS] two submissions rejected
  [PASS] top error patterns present

=== RESULTS: 29 passed, 0 failed ===
All tests passed.
~/Documents/Duke/26Summer/AIPI_561/aipi-561/week7 main* ❯
```

6. What I would do next

The natural next step is to wire `CostAnalyzer` into the live `query()` path so every real call records its component costs, and to apply a validated correction back into the agent (for example as a retrieval-time override for the corrected query). Both are straightforward given that the interfaces already line up; I kept them offline here so the deliverable is reproducible without quota.